

Serverless Container Deployment on AWS

(using Docker, Amazon ECR, ECS & Fargate)

1. Project Overview

In this project, I **containerized my portfolio website using Docker**, tested it on an **EC2 instance**, and pushed the Docker image to **Amazon ECR**. Then, I deployed the image using **Amazon ECS with AWS Fargate**.

With **Fargate**, AWS runs the container without requiring me to manage the underlying servers, making the final deployment **serverless**.

2. Objectives

- ✓ Containerize my **portfolio website using Docker and Apache**.
- ✓ Test the Docker container on an **EC2 instance**.
- ✓ Push and store the Docker image in **Amazon ECR**.
- ✓ Create an **ECS Task Definition** using the ECR image.
- ✓ Deploy the container using **ECS with AWS Fargate**.
- ✓ Configure the required **networking and security group rules**.
- ✓ Verify that the portfolio website is **running and publicly accessible**.

3. AWS Services and Technologies Used

Technology / Service	Purpose
Docker	Created the container image of the portfolio.
Apache	Hosted the portfolio website inside the container.
Amazon EC2	Built and tested the Docker container.
AWS IAM	Provided permissions to access ECR.
Amazon ECR	Stored the Docker image.
Amazon ECS	Managed and deployed the container.
AWS Fargate	Ran the container without managing servers.
Security Groups	Controlled network access to the application.

4. Architecture and Workflow

Portfolio Code + Dockerfile



Docker Image



Amazon ECR



Amazon ECS



AWS Fargate

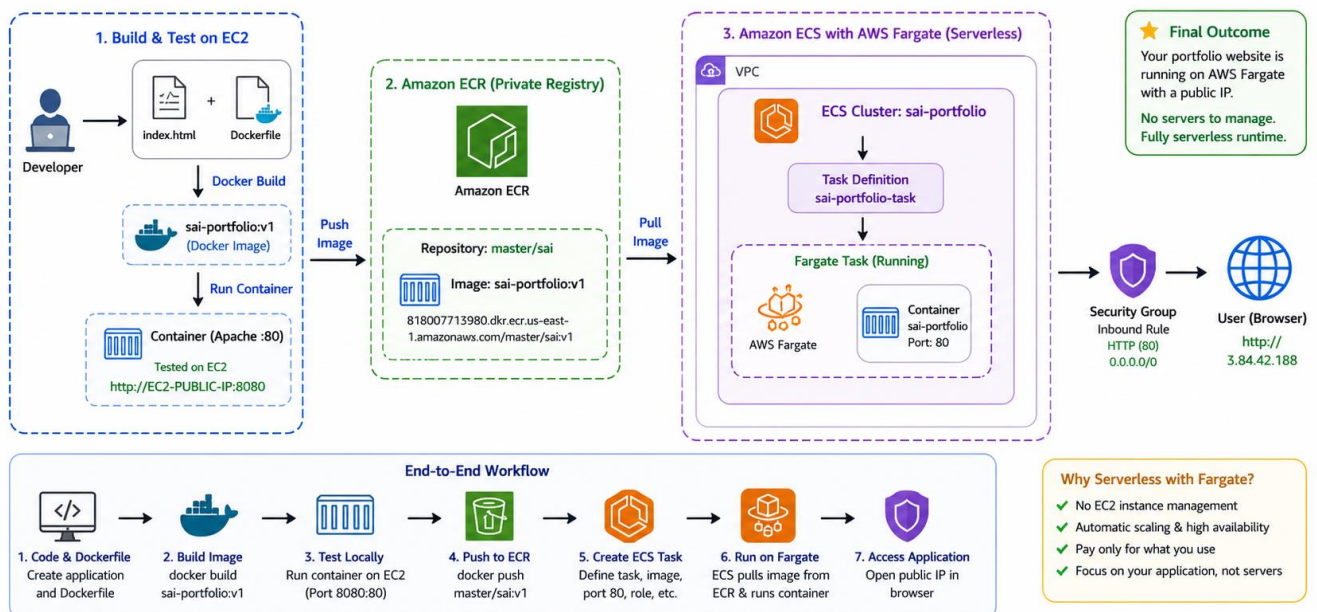


User

The portfolio was first **built and tested as a Docker container on EC2**. The Docker image was then pushed to **Amazon ECR**. Finally, **ECS pulled the image from ECR and deployed it using AWS Fargate**, making the portfolio publicly accessible without managing a server for the final deployment.

Serverless Container Deployment on AWS

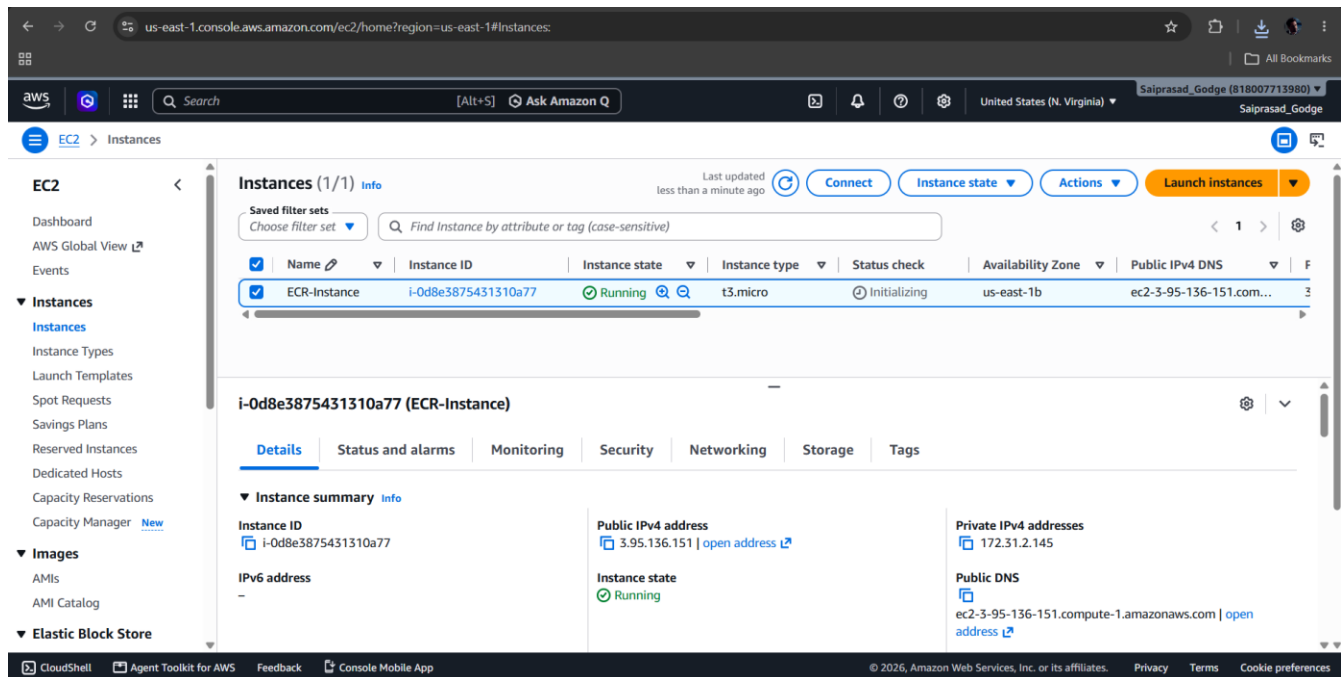
Using Docker, Amazon ECR, Amazon ECS & AWS Fargate



Step-by-Step Implementation Guide:

Step 1: Launch an Amazon Linux EC2 Instance

First, create an **Amazon EC2 instance** that will be used to build and test the Docker application.



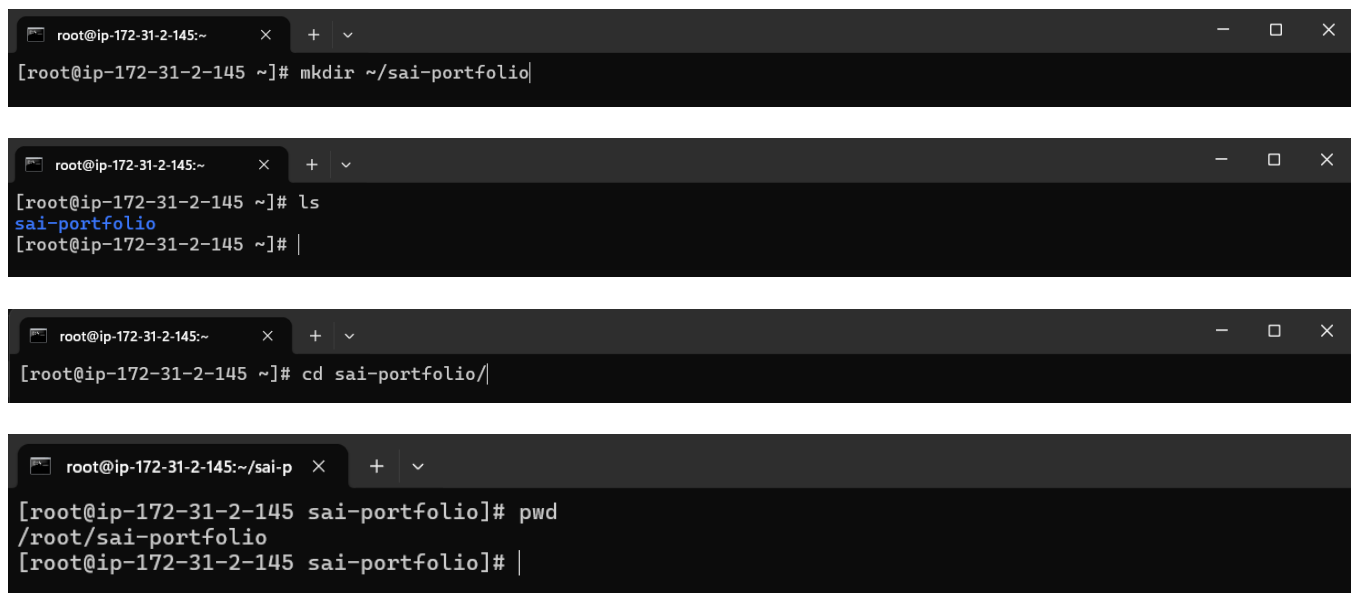
Step 2: Install and Start Docker

```
sudo dnf install docker -y
```

```
sudo systemctl enable --now docker
```

```
sudo systemctl status docker
```

Step 3: Create the Portfolio Project Directory



Step 4: Create the Portfolio Website

Create the index.html file:

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# vi index.html
```

View Portfolio Code on GitHub:

<https://github.com/SAI-GODGE/serverless-container-webapp/blob/main/index.html>

Step 5: Create the Dockerfile

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# vi Dockerfile
```

View Dockerfile on GitHub:

<https://github.com/SAI-GODGE/serverless-container-webapp/blob/main/Dockerfile>

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# ls -l
total 8
-rw-r--r--. 1 root root 163 Jul 28 04:24 Dockerfile
-rw-r--r--. 1 root root 3550 Jul 28 04:23 index.html
[root@ip-172-31-2-145 sai-portfolio]# |
```

Step 6: Check the status of Docker

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 04:01:12 UTC; 24min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 27390 (dockerd)
      Tasks: 9
     Memory: 29.4M
        CPU: 612ms
    CGroup: /system.slice/docker.service
            └─27390 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768

Jul 28 04:01:11 ip-172-31-2-145.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine.
Jul 28 04:01:11 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:11.570081412Z" level=info msg="Star
Jul 28 04:01:11 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:11.642571001Z" level=info msg="Load
Jul 28 04:01:12 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:12.143810134Z" level=info msg="Load
Jul 28 04:01:12 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:12.176905174Z" level=info msg="Dock
Jul 28 04:01:12 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:12.177047673Z" level=info msg="Daem
Jul 28 04:01:12 ip-172-31-2-145.ec2.internal dockerd[27390]: time="2026-07-28T04:01:12.218210792Z" level=info msg="API
Jul 28 04:01:12 ip-172-31-2-145.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-20/20 (END)
```

Step 7: Build the Docker Image

`docker build -t sai-portfolio:v1 .`

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# docker build -t sai-portfolio:v1 .
[+] Building 33.7s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 262B
=> [internal] load metadata for docker.io/library/amazonlinux:2023
=> [internal] load .dockerignore
=> => transferring context: 28
=> [1/3] FROM docker.io/library/amazonlinux:2023@sha256:c4b0d324eaacbc5805ab5f147ba71963184752da90340bf0803c1034
=> => resolve docker.io/library/amazonlinux:2023@sha256:c4b0d324eaacbc5805ab5f147ba71963184752da90340bf0803c1034
=> => sha256:75f3103684966a2cc4d352ad4818811a6f9292cfb230afd6467c087975657503 1.02kB / 1.02kB
=> => sha256:c4f7cd018991ef5ca29b3dca7ad7ade11f8f65516c48919df080d7a13a62de29 586B / 586B
=> => sha256:5492e2c54dedbfa8cbd57605d23f9468dae3f869273566ed212185e48340d5d5 54.57MB / 54.57MB
=> => sha256:c4b0d324eaacbc5805ab5f147ba71963184752da90340bf0803c1034a170c8d1 2.38kB / 2.38kB
=> => extracting sha256:5492e2c54dedbfa8cbd57605d23f9468dae3f869273566ed212185e48340d5d5
=> [internal] load build context
=> => transferring context: 3.65kB
=> [2/3] RUN dnf install -y httpd && dnf clean all
=> [3/3] COPY index.html /var/www/html/index.html
=> exporting to image
=> => exporting layers
=> => writing image sha256:4ff11248cf3c98c034b9d2a46df118ecce982f6fad679c17580b94d913f510d8
=> => naming to docker.io/library/sai-portfolio:v1
[root@ip-172-31-2-145 sai-portfolio]#
```

docker images

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
sai-portfolio        v1             4ff11248cf3c   38 seconds ago  218MB
[root@ip-172-31-2-145 sai-portfolio]#
```

Step 8: Run and test the Docker Container

docker run -d --name sai-portfolio -p 8080:80 sai-portfolio:v1

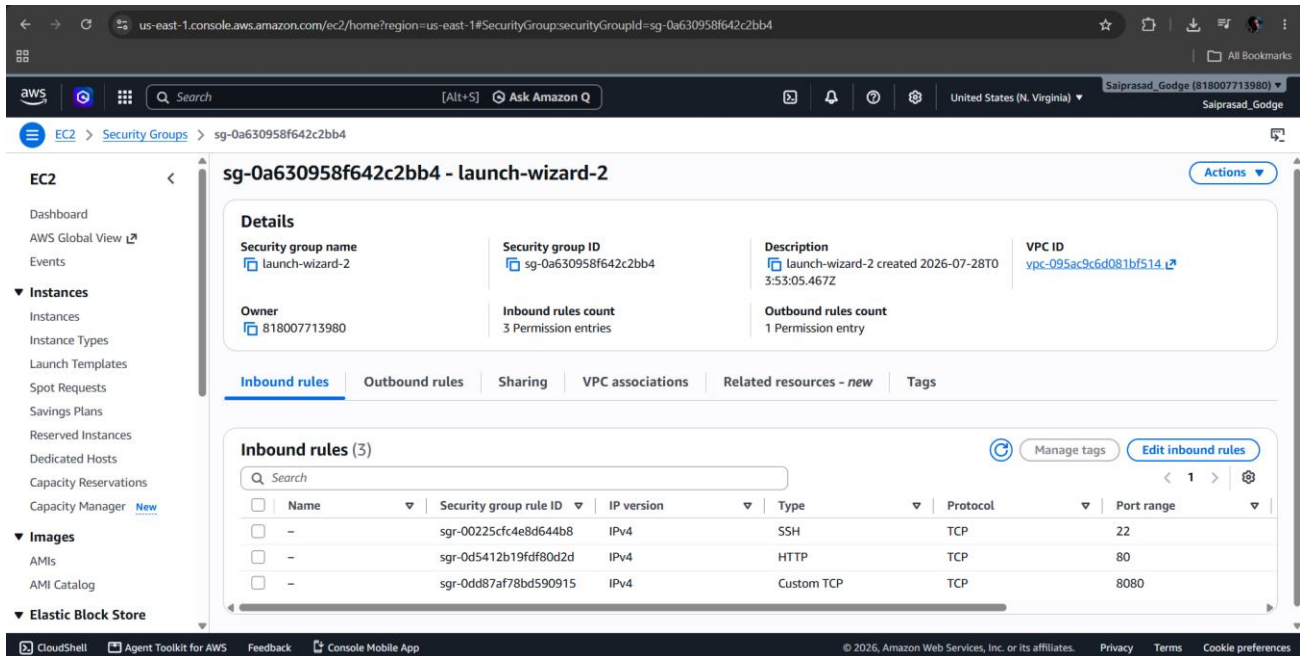
```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# docker run -d --name sai-portfolio -p 8080:80 sai-portfolio:v1
83b8d3eee52e46ea5e138cc3411740794799e8f28f75938df78f65213e635fc0
[root@ip-172-31-2-145 sai-portfolio]#
```

docker ps

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
83b8d3eee52e   sai-portfolio:v1 "/usr/sbin/httpd -D ..." 51 seconds ago Up 50 seconds 0.0.0.0:8080->80/tcp, :::8080->80/tcp
[root@ip-172-31-2-145 sai-portfolio]#
```

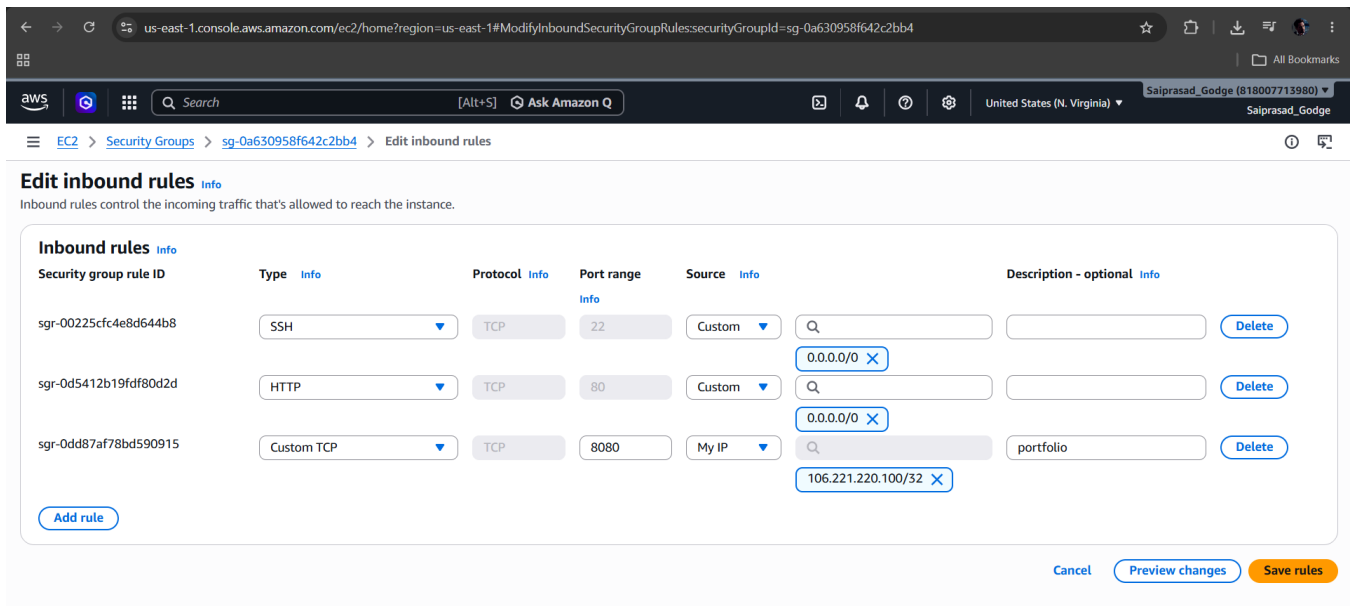
Step 9: Review the EC2 Security Group

Configured the EC2 Security Group to allow SSH (22), HTTP (80), and port 8080 for accessing the Docker portfolio.



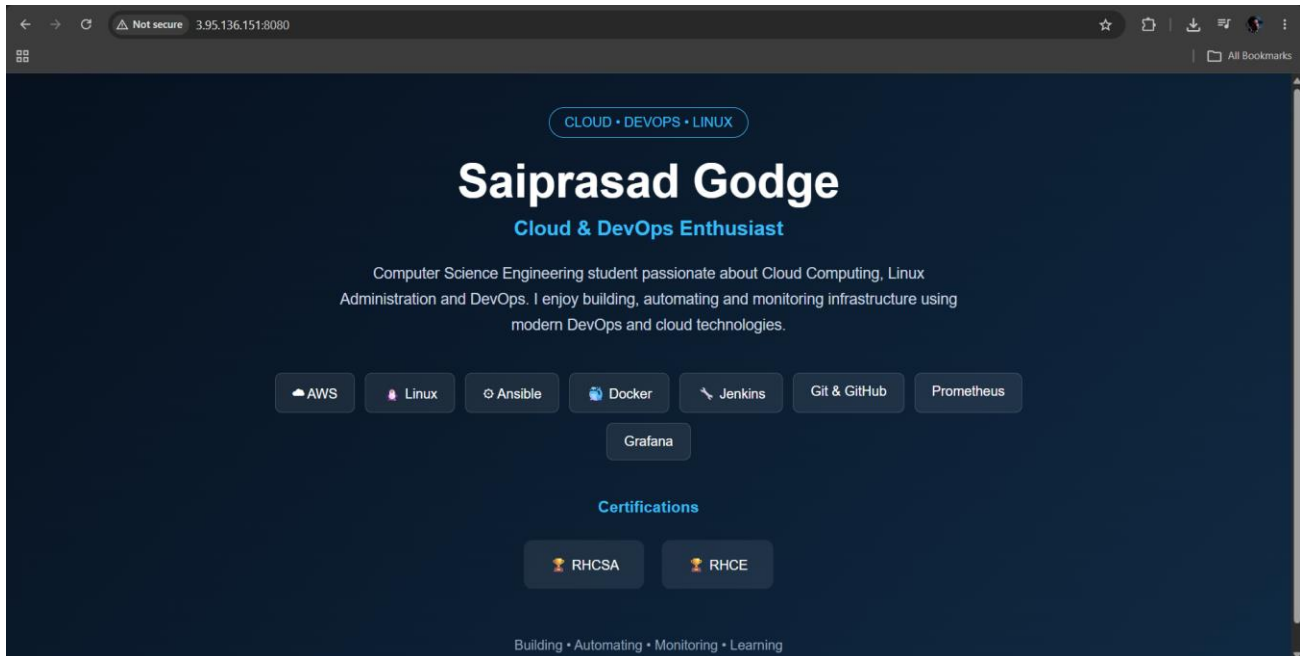
Step 10: Allow Portfolio Access on Port 8080

Added a Custom TCP inbound rule for port 8080 to allow access to the Docker portfolio from the browser.



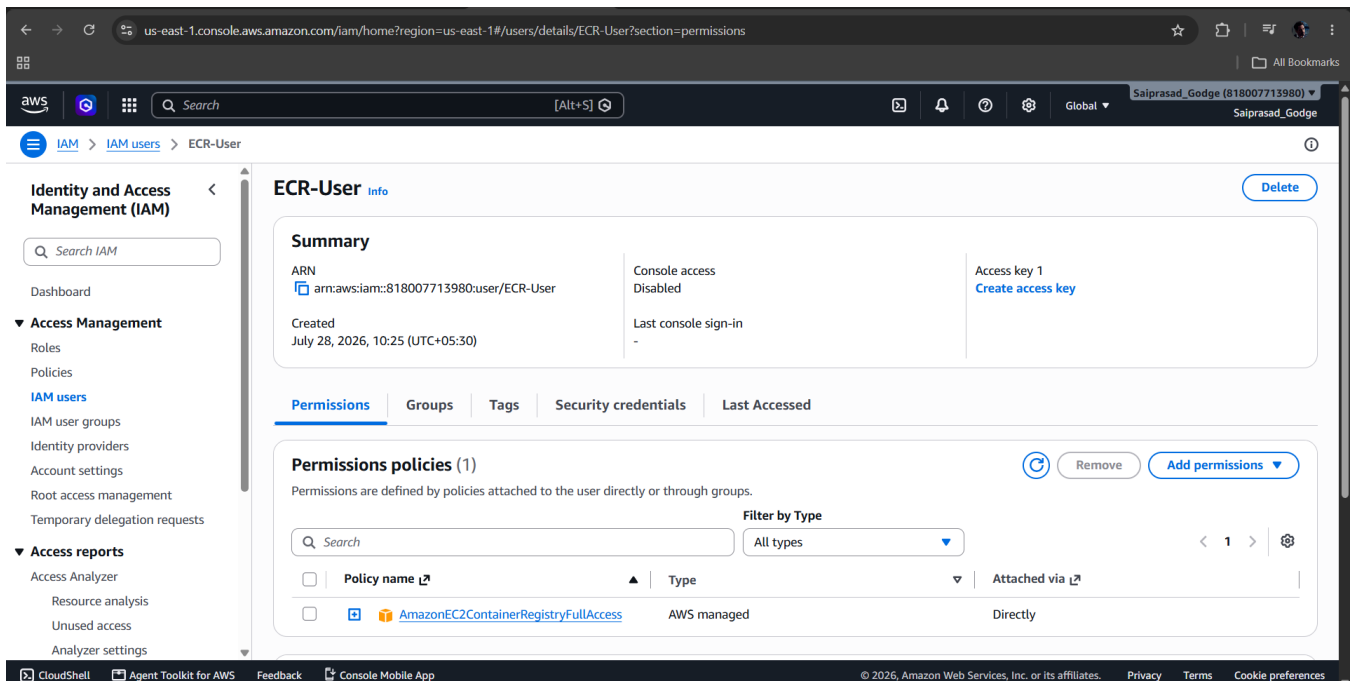
Step 11: Portfolio Running from the Container

The portfolio was successfully accessed using the EC2 public IP on port 8080, confirming that the Docker container was working correctly.



Step 12: IAM Permission for Amazon ECR

Created an **IAM user (ECR-User)** and attached the **AmazonEC2ContainerRegistryFullAccess** policy to access and push Docker images to **Amazon ECR** using AWS CLI.



Step 13: Create an Access Key

Created an **Access Key** for **ECR-User** to authenticate and access AWS services through the **AWS CLI**.

The screenshot shows the AWS IAM console interface. The breadcrumb navigation at the top reads: IAM > IAM users > ECR-User > Create access key. On the left, a sidebar shows a three-step process: Step 1 (Access key best practices & alternatives), Step 2 (optional, Set description tag), and Step 3 (Retrieve access keys, which is the active step). The main content area is titled 'Retrieve access keys' and includes an 'Access key' section with a warning: 'If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.' Below this, there are two fields: 'Access key' containing 'AKIA345IPFC6AX4M7P6Q' and 'Secret access key' containing a masked value with a 'Show' button. An 'Access key best practices' section follows, listing guidelines like 'Never store your access key in plain text' and 'Enable least-privilege permissions'. At the bottom right, there are 'Download .csv file' and 'Done' buttons. The footer contains links for CloudShell, Agent Toolkit for AWS, Feedback, and Console Mobile App, along with copyright information for 2026 Amazon Web Services, Inc.

Step 14: Verify AWS CLI Installation

Verified that **AWS CLI version 2** is installed and available on the Amazon Linux instance using the **aws --version** command.

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.38-76.139.amzn2023.x86_64 source/x86_64.amzn.2023
[root@ip-172-31-2-145 sai-portfolio]# |
```

Step 15: Verify the AWS CLI Identity

Used `aws sts get-caller-identity` to confirm that AWS CLI is successfully authenticated as **ECR-User**.

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# aws sts get-caller-identity
{
  "UserId": "AIDA345IPFC6MVYFYBJVS",
  "Account": "818007713980",
  "Arn": "arn:aws:iam::818007713980:user/ECR-User"
}
[root@ip-172-31-2-145 sai-portfolio]# |
```

Step 16: Authenticate Docker with Amazon ECR

Authenticated **Docker with Amazon ECR** using AWS CLI. The **Login Succeeded** message confirms successful ECR access.

```
root@ip-172-31-2-145:~/sai-p x + v
[root@ip-172-31-2-145 sai-portfolio]# aws ecr get-login-password --region us-east-1 | docker login --username AWS --pass
word-stdin 818007713980.dkr.ecr.us-east-1.amazonaws.com/master/sai
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-2-145 sai-portfolio]# |
```


Step 17: Tag the Docker Image for ECR

Tagged the local **sai-portfolio:v1** Docker image with the **Amazon ECR repository URI** so it can be pushed to ECR.

```
root@ip-172-31-2-145:~/sai-p [root@ip-172-31-2-145 sai-portfolio]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
sai-portfolio        v1           4fff11248cf3c    46 minutes ago   218MB
[root@ip-172-31-2-145 sai-portfolio]# docker tag sai-portfolio:v1 818007713980.dkr.ecr.us-east-1.amazonaws.com/master/sai:v1
[root@ip-172-31-2-145 sai-portfolio]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
818007713980.dkr.ecr.us-east-1.amazonaws.com/master/sai  v1           4fff11248cf3c    46 minutes ago   218MB
sai-portfolio        v1           4fff11248cf3c    46 minutes ago   218MB
[root@ip-172-31-2-145 sai-portfolio]# |
```

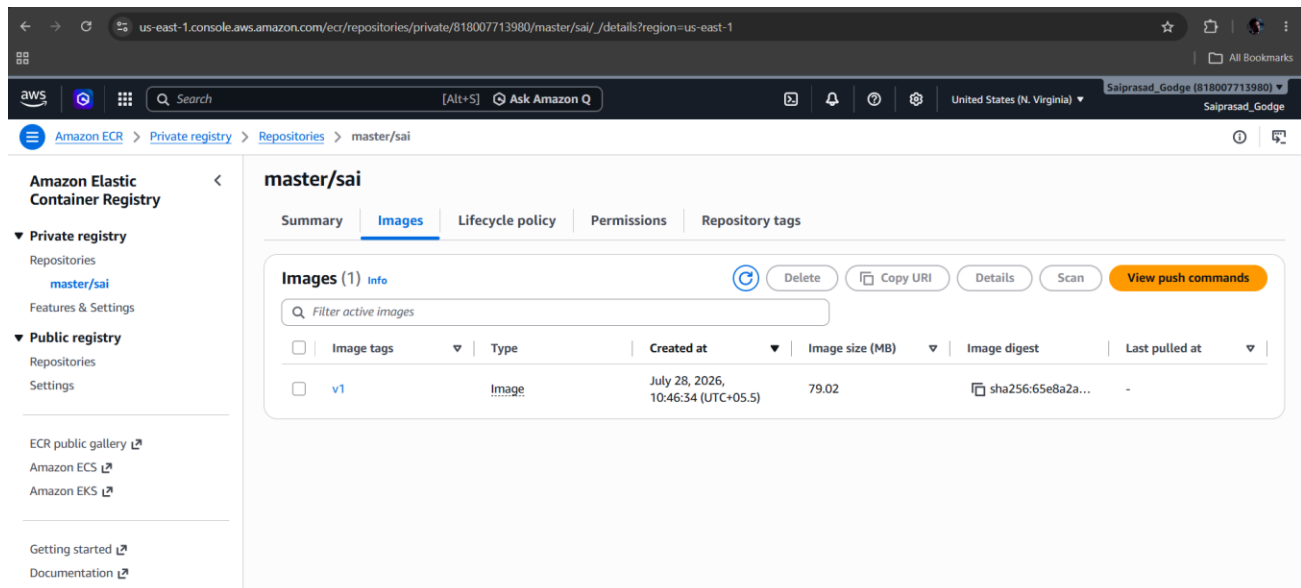
Step 18: Push the Docker Image to Amazon ECR

Pushed the tagged **sai-portfolio:v1** Docker image successfully to the private **Amazon ECR repository**.

```
root@ip-172-31-2-145:~/sai-p [root@ip-172-31-2-145 sai-portfolio]# docker push 818007713980.dkr.ecr.us-east-1.amazonaws.com/master/sai:v1
The push refers to repository [818007713980.dkr.ecr.us-east-1.amazonaws.com/master/sai]
8b2a84e84e42: Pushed
7d6930263f61: Pushed
6f6f5bad5554: Pushed
v1: digest: sha256:65e8a2a149ed72b59d54d82f73c8afaf79f0c19d34d4e278dbf9fa424b2fb99c size: 949
[root@ip-172-31-2-145 sai-portfolio]# |
```

Step 19: Verify the Image in Amazon ECR

Verified that the **sai-portfolio:v1** Docker image is successfully stored in the private **master/sai ECR repository**, ready for ECS deployment.

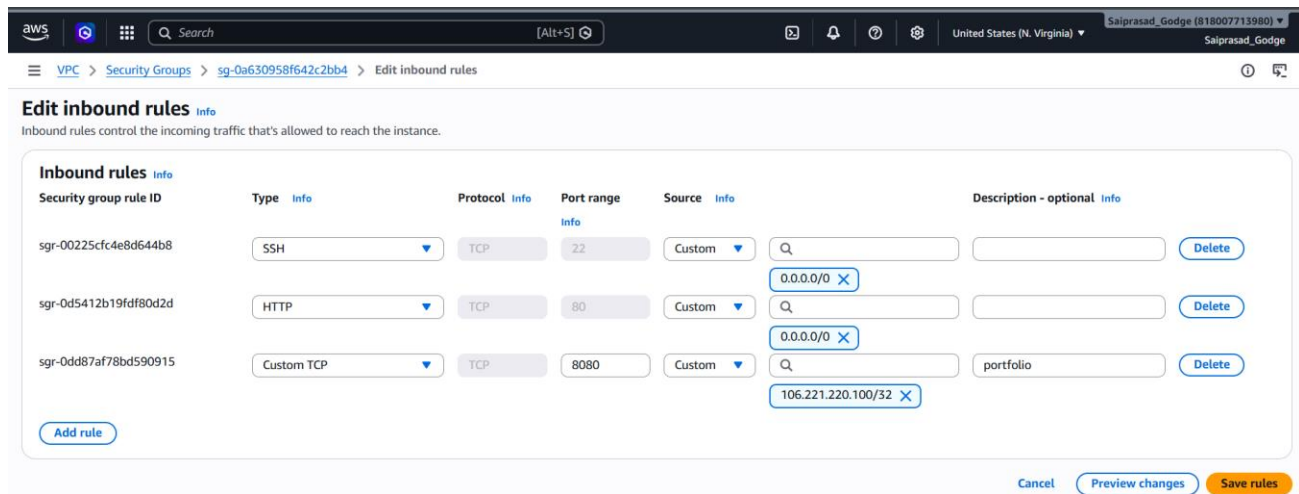


Step 20: Configure Security Group

Configured the **Security Group** to allow **HTTP traffic on port 80**, enabling users to access the portfolio running on ECS Fargate.

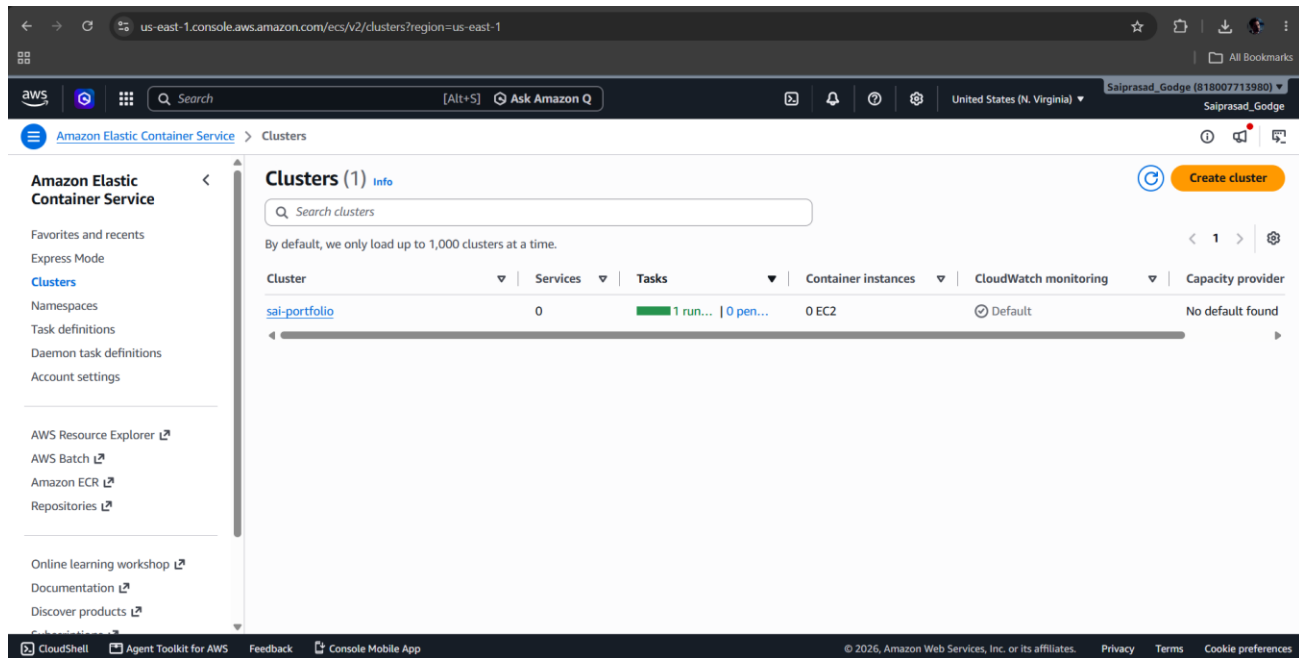
EC2 testing: Port 8080 → Docker container port 80

Fargate deployment: Port 80 → Container port 80



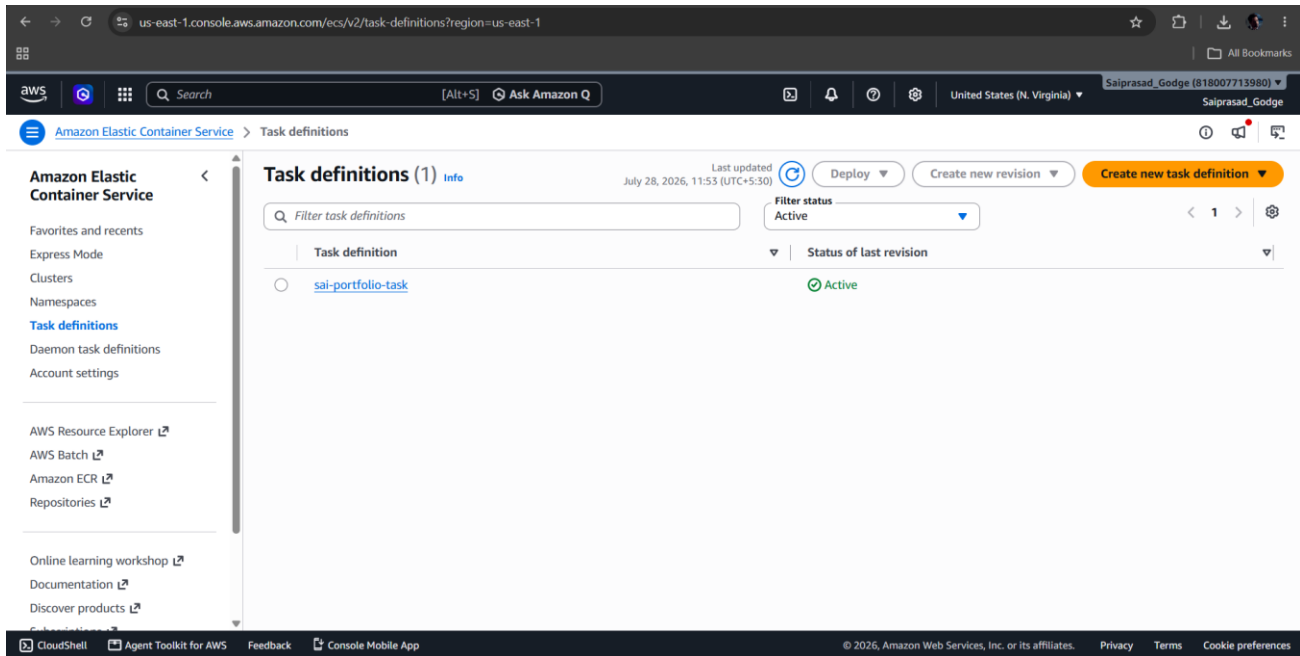
Step 21: Create an Amazon ECS Cluster

Created an ECS cluster named sai-portfolio to manage and run the containerized portfolio application using AWS Fargate.



Step 22: Create ECS Task Definition

Created the **sai-portfolio-task** Task Definition with the **ECR image**, **Fargate compute resources**, **networking**, and **container port 80** configuration.

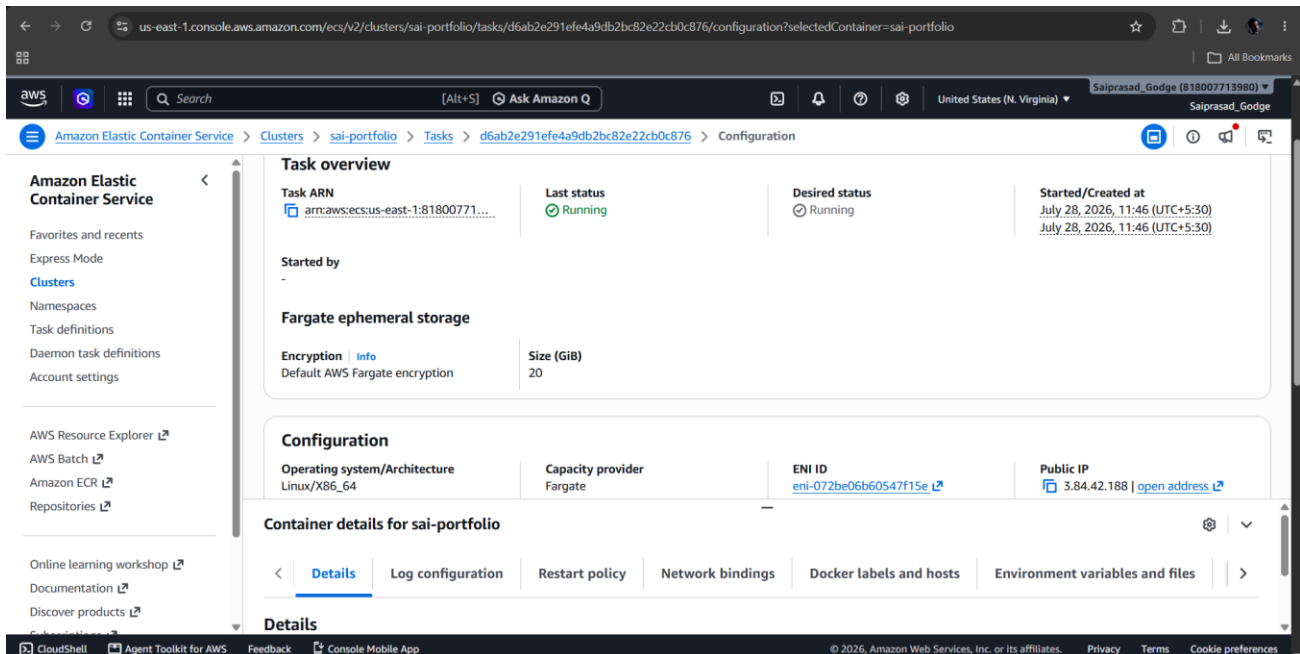


Step 23: Run the Task Using AWS Fargate

Deployed the **sai-portfolio-task** using **AWS Fargate**. The task reached **Running** status and received a **public IP** to access the portfolio.

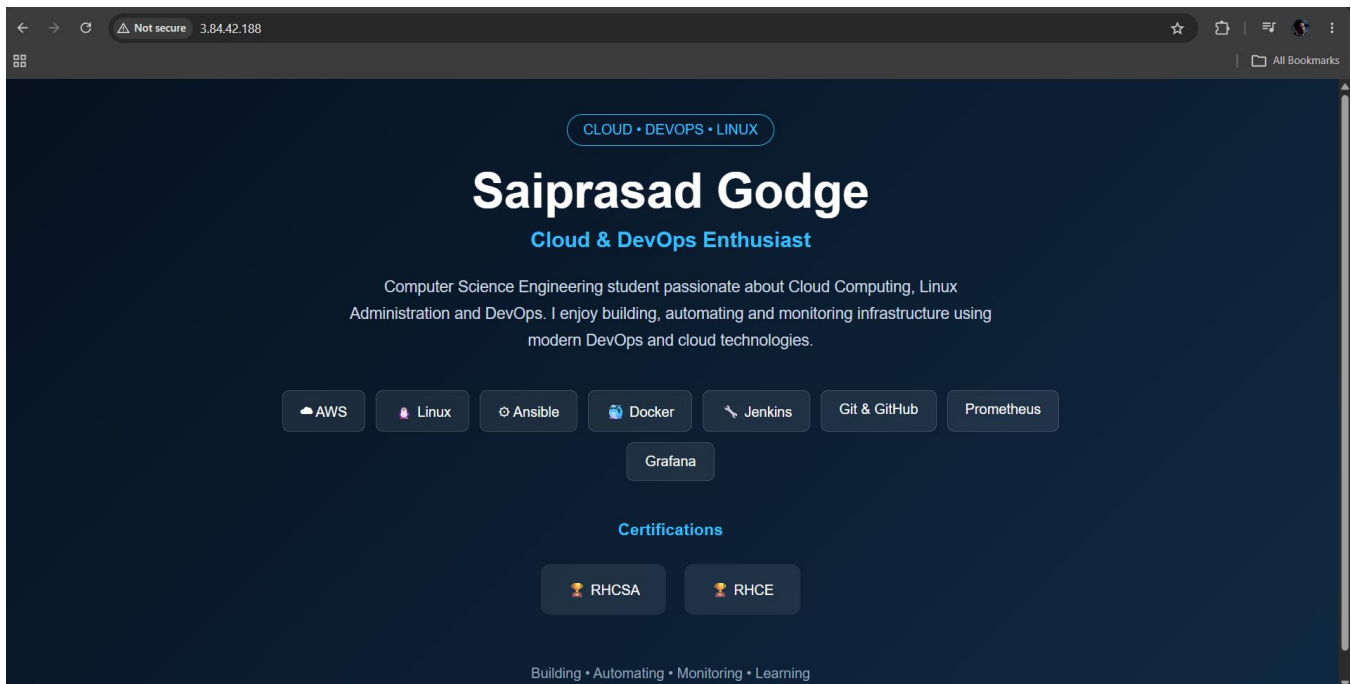
Step 24: Verify ECS Cluster and Fargate Task

Verified that the **sai-portfolio** ECS cluster has a **running Fargate task**, confirming that the container is successfully deployed without managing an EC2 server.



Step 25: Access the Portfolio Using the Fargate Public IP

The portfolio was successfully accessed using the public IP of the AWS Fargate task, confirming that the containerized application was successfully deployed and is publicly accessible through Amazon ECS with AWS Fargate.



5. Why This Is Called a Serverless Container Deployment

The final application runs on AWS Fargate, where AWS manages the underlying servers for us. Therefore, we can deploy and run our container without managing EC2 instances, making it a serverless container deployment.

Note: EC2 was only used to build and test the Docker image.

6. Skills Demonstrated

Docker • Amazon Linux • Apache • AWS CLI • IAM • Amazon ECR • Amazon ECS • AWS Fargate • VPC • Security Groups • Container Deployment • Serverless Computing